Ho Chi Minh City University of Technology (HCMUT), VNU-HCM

Faculty of Computer Science and Engineering



ASSIGNMENT 2

Deep Learning on Large-Scale Data and Specialized Tasks

Fine-Grained Food-101 Classification: A From-Scratch CNN Baseline versus a Fine-Tuned ImageNet ResNet-50

Course:	CO3133 – Deep Learning and Its Applications
Instructor:	Lê Thành Sách
Semester:	261
Group:	G-03
Task track:	Image classification
Dataset:	Food-101
Members:	Nguyen Van An (2352013)
	Vo Le Hai Dang (2352257)
	Huynh Vuong Khang (2350011)

Ho Chi Minh City, November 2026

1 Problem and Data Description

1.1 Problem Formulation

Food image recognition supports dietary assessment, menu retrieval, and automated nutrition logging. The task-defining property of Food-101 is that the label is determined by the ingredients and the preparation of a dish rather than by a coarse visual category: dishes that differ only in cut, garnish, or cooking method are visually close, while one class covers large variation in plating, lighting, and camera angle.

- **Input:** one RGB photograph $x \in \mathbb{R}^{3\times H\times W}$ of a single dish, resized to $H = W = 128$ (baseline) or $H = W = 224$ (ResNet-50).
- **Output:** a class index $\hat{y} = \arg\max_k f_\theta(x)_k$ over the 101 Food-101 dish categories, computed from the logits $f_\theta(x) \in \mathbb{R}^{101}$.
- **Prediction unit:** one photograph.
- **Objective:** minimise the categorical cross-entropy over the training set $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$,

$$\mathcal{L}(\theta) \;=\; -\frac{1}{N}\sum_{i=1}^{N} \log \frac{\exp(z_{i,y_i})}{\sum_{k=1}^{101}\exp(z_{i,k})}, \qquad z_i = f_\theta(x_i), \tag{1}$$

  with raw logits passed to the loss (no softmax applied to the model output).
- **Evaluation metrics:** top-1 accuracy $\mathrm{Acc} = \frac{1}{N}\sum_i \mathbb{1}[\hat{y}_i = y_i]$ and the macro-averaged $F_1 = \frac{1}{101}\sum_{k=1}^{101}\frac{2P_kR_k}{P_k+R_k}$, reported together with parameter count, training time, and inference time.
- **Main difficulties:** fine-grained inter-class similarity between semantically overlapping classes (`steak`, `filet_mignon`, `prime_rib`); label noise in the official training split; background clutter that does not identify the dish; and one dish per image, so plates holding several dishes are labelled ambiguously.

1.2 Dataset Description

- **Name and source:** Food-101, ETH Zürich, collected from Foodspotting.com. Downloaded from the HuggingFace parquet mirror `ethz/food101`, which reproduces the official release; the ETH Zürich dataset page is the authoritative reference for the split definition.
- **Version:** the original 2014 release. The dataset is static and has no later version.
- **Licence (verbatim from the dataset card):** *"The Food-101 data set consists of images from Foodspotting which are not property of ETHZ. Any use beyond scientific fair use must be negotiated with the respective picture owners."* The use in this project is scientific fair use within a university course.
- **Collection and labelling:** images and dish names were mined from Foodspotting. The authors state that the 750 training images per class carry automatically assigned labels that were intentionally not cleaned, whereas the 250 test images per class were manually verified.
- **Sample counts and annotations:** 101,000 images, of which 75,750 are training and 25,250 are test. Each image carries exactly one dish label; there are no bounding boxes, segmentation masks, ingredient labels, or other annotations.
- **Class distribution:** 101 classes, exactly 750 training and 250 test images per class; the dataset is balanced by construction.
- **Storage size and format:** approximately 10 GB of JPEG images, distributed as 11 parquet shards by the mirror. The data are excluded from the repository and downloaded on demand.
- **Bias and limitations:** the 101 categories are a Western-centric selection and do not cover Vietnamese or most regional cuisines; each image contains one dish, so mixed plates are ambiguous; background context correlates with the class; and the training labels contain

documented label noise, which limits the validation accuracy attainable when validation is drawn from the training split.

1.3 Exploratory Data Analysis

The analysis is implemented in `assignment2/notebooks/eda.ipynb` and operates on the official training split at 512×512 resolution.

- **Class distribution and imbalance.** After duplicate removal the per-class count ranges from 666 to 675 images; every class except one lies between 671 and 675. Accuracy and macro-F1 therefore weight the classes almost equally, and no class re-weighting, resampling, or oversampling was applied.
- **Input size.** Images are JPEG files of variable resolution and variable aspect ratio with a maximum side of 512 px. Because the aspect ratio is not fixed, the pipeline uses `RandomResizedCrop` for training and resize-then-centre-crop for evaluation instead of a plain resize.
- **Colour distribution.** The measured RGB statistics of the cleaned training split are mean $= (0.5450,\ 0.4436,\ 0.3437)$ and standard deviation $= (0.2720,\ 0.2746,\ 0.2793)$ over the $[0, 1]$ range. The red channel has the highest mean, the blue channel has a concentration of near-zero intensities, and the HSV analysis shows warm hues dominating with saturation and value broadly spread rather than concentrated in a narrow band.
- **Representative samples.** A random sample of training images shows photographed food with substantial background clutter; in several images the plate or the table occupies more area than the dish.
- **Near-duplicate detection.** A 64-bit perceptual hash (pHash) was computed for all 101,000 images and compared within each class at a Hamming-distance threshold of 10. The search returned 271 candidate pairs: 160 train–train, 9 test–test, and 102 train–test. From the train–train pairs, 162 training images were selected for removal, giving the 75,588-image training set used in the EDA notebook (see §1.4).

1.4 Data Splitting

Table 1 summarises the three splits.

Table 1: Data splits. The official test split is used unchanged; the validation split is carved from the official training split.

Split	Images	Images per class	Origin
Train (optimisation)	68,175	≈675	official train minus validation
Validation (model selection)	7,575	≈75	stratified 10% of official train
Test (final evaluation)	25,250	250	official test split, unchanged
Total	101,000	—	—

- **Split type:** the official train/test split is retained as released. The test split was never re-split, and no test image participates in optimisation or in model selection.
- **Split seed:** 42 for the train/validation carve, identical for every model and every training seed.
- **Stratification:** the validation carve is stratified by class (`sklearn.model_selection.train_test_split` with `stratify` set to the label vector), preserving the per-class ratio of the official training split.
- **Split unit:** one photograph. Both splits are disjoint image sets defined by index, so no image appears in more than one split.

- **Leakage prevention:** the index-level split excludes exact duplicates across splits. The pHash analysis quantified the near-duplicates that an index-level split cannot remove: 102 train–test pairs, i.e. at most 102 of the 25,250 test images (0.40%) have a near-duplicate in the training set. The removal list derived from that analysis (162 training images) was applied in the EDA notebook, which operates on 75,588 training images; the training runs reported in this document use the unmodified official split, so at most 0.40% of the test images can be affected by near-duplicate leakage.
- **Subset selection:** none. The full official split is used; no per-class capping, downsampling, or filtering was applied.

1.5 Preprocessing

- **Cleaning:** images are decoded as JPEG and converted to RGB. The release contains a small number of truncated JPEG files that emit `Truncated File Read` warnings on decode; this specific warning is filtered in the loader so that a damaged file does not interrupt an epoch. No image was discarded for being damaged.
- **Invalid-annotation handling:** no annotation-level filtering was applied. The training labels are used as released, including their documented noise, so that the reported numbers correspond to the standard benchmark protocol.
- **Resize and normalisation:** images use ImageNet normalisation with $\mu = (0.485, 0.456, 0.406)$ and $\sigma = (0.229, 0.224, 0.225)$, matching the pretrained initialisation.
- **Augmentation (training only):** `RandomResizedCrop(size, scale=(0.7,1.0))` followed by `RandomHorizontalFlip()`. Evaluation uses `Resize(round(size·256/224))` followed by `CenterCrop(size)`. The same policy is applied to every configuration, so augmentation is not a varying factor. No colour jitter, rotation, mixup, cutmix, or random erasing is used.
- **Input tensor shapes:** the baseline consumes $3 \times 128 \times 128$ tensors and both ResNet-50 configurations consume $3 \times 224 \times 224$ tensors, in NCHW layout with dtype `float32`.
- **One batch after preprocessing:** the pipeline was verified with a live batch, which returned `classes: 101`, `batch: (8, 3, 128, 128) torch.float32` and `labels: [26, 76, 60, 7, 46, 13, 34, 92]`.

2 Methodology

2.1 Pipeline Overview

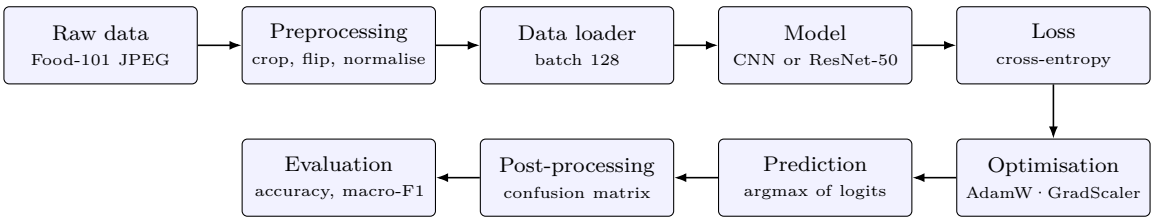


Figure 1: End-to-end pipeline, shared by all three configurations. Two things vary. Preprocessing uses 128×128 for the baseline and 224×224 for both ResNet-50 configurations. At the optimisation stage the baseline and `resnet50_full` train all parameters, whereas `resnet50_frozen` trains only its 206,949-parameter head and holds every BatchNorm layer in evaluation mode. The chain runs for 30 epochs; the checkpoint with the best validation macro-F1 is then restored and the test split is evaluated once.

2.2 Baseline: Self-Implemented CNN

- **Architecture:** a convolutional feature extractor followed by a multilayer-perceptron head, reusing the CNN implemented for Assignment 1 (`assignment1/src/models/cnn.py`, wrapped by `assignment2/src/models/baseline.py`).
- **Feature extractor:** four blocks of [$\text{Conv}_{3\times3}$, ReLU, MaxPool_2] with 32, 64, 128, 256 output channels, `stride`=1 and `padding`=1, which maps 128×128 to $256 \times 8 \times 8$.
- **Head:** the feature map is flattened to 16,384 dimensions and passed to one linear layer of 512 units with ReLU followed by a linear layer of 101 outputs. Dropout is disabled ($p = 0$), so no dropout regularisation is applied.
- **Input representation:** flattened spatial feature maps produced by the convolutional stack, not raw pixels.
- **Loss:** `nn.CrossEntropyLoss` on raw logits.
- **Metric:** top-1 accuracy and macro-F1, identical to all other configurations.
- **Parameter count:** 8,829,349, all trainable. Of these, 8,440,933 (95.6%) are in the two head layers (16384×512 and 512×101).
- **Strengths:** trained from random initialisation on the target dataset only, so it measures what the task yields with 68k images and no external data; small memory footprint; no dependency on a pretrained checkpoint.
- **Limits:** no external prior; flattening after the last pooling stage discards spatial layout; no residual connections; 95.6% of the parameters are in dense layers rather than in the convolutional representation.
- **Implementation origin:** the network, the training loop, the Food-101 dataset adaptor, and the evaluation and plotting utilities are self-implemented in this repository. The libraries used are PyTorch for tensors, autograd, and optimisation; `torchvision` for image transforms; `scikit-learn` for the stratified split and for the accuracy, macro-F1, confusion-matrix, and per-class precision/recall/F_1 computations; `HuggingFace datasets` and `huggingface_hub` for the Food-101 download; and `numpy`, `pandas`, and `matplotlib` for aggregation and figures.

2.3 Main Model: Pretrained ResNet-50 and Fine-Tuning Protocol

- **Architecture:** ResNet-50 from `torchvision.models`, initialised with `ResNet50_Weights.IMAGENET1K_V2`, i.e. with ImageNet weights. The final layer of 1000 outputs is replaced by a new `Linear(2048, 101)` layer; all other layers are unchanged.
- **Input representation:** 224×224 RGB images normalised with the ImageNet statistics, i.e. the input distribution of the pretrained weights.
- **Loss:** `nn.CrossEntropyLoss`, identical in both configurations.
- **Fine-tuning protocol, `resnet50_full`:** all 23,714,981 parameters are trainable. A single global learning rate of 10^{-4} is applied to the whole network, with no discriminative or layer-wise learning rates; AdamW with weight decay 10^{-4}; 30 epochs at batch size 128; mixed precision enabled (CUDA autocast with a gradient scaler).
- **Fine-tuning protocol, `resnet50_frozen`:** the same model and the same data, augmentation, optimiser, learning rate, batch size, epoch budget, and seeds, but every backbone parameter (23,508,032) has `requires_grad=False` and only the 206,949 parameters of the new head are optimised. BatchNorm running statistics are additionally frozen by placing all `BatchNorm2d` modules in evaluation mode for the whole training phase, so the backbone acts as a fixed feature extractor.
- **Parameter count:** 23,714,981 in both configurations.
- **Strengths:** transfers ImageNet features and converges within a few epochs; the frozen variant isolates the contribution of backbone adaptation at identical model size.
- **Limits:** requires a pretrained checkpoint and therefore external data; the parameter count

is 2.7 times that of the baseline; the fidelity of the frozen variant depends on how well the ImageNet feature space is linearly separable for the target classes.

- **Implementation origin:** the network definition and the pretrained weights come from `torchvision`; the head replacement, the backbone freezing, the BatchNorm handling, the training loop, and the evaluation are self-implemented in this repository.

2.4 Experimental Design

Table 2: Specification of the controlled experiment (handbook §20.4 and §21).

Field	Specification
Hypothesis	Full fine-tuning of an ImageNet ResNet-50 reaches higher test accuracy and macro-F1 than training only a linear head on a frozen backbone, at equal model size, data, augmentation, optimiser, learning rate, batch size, and epoch budget.
Changed factor	Backbone parameters frozen versus fully trainable.
Fixed factors	Dataset and split (seed 42), augmentation policy, input resolution (224), loss, optimiser (AdamW), learning rate (10^{-4}), weight decay (10^{-4}), batch size (128), epoch budget (30), training seeds (42, 67, 69), hardware, and mixed-precision setting.
Metrics	Test accuracy and test macro-F1 (primary); validation accuracy and validation macro-F1; training time and inference time per image (secondary).
Decision criterion	Compare the mean over the three seeds. The hypothesis is supported if the full fine-tuning mean exceeds the frozen mean by more than the across-seed standard deviation of both configurations.

The baseline is a reference configuration, not a controlled factor of the ablation: it differs from both ResNet-50 configurations in initialisation, input resolution (128 versus 224), learning rate (10^{-3} versus 10^{-4}), and parameter count. Its role is to establish the accuracy attainable without pretraining; it is not interpreted as an isolated architectural comparison.

2.5 Training Setup

Three training seeds are run per configuration, giving nine runs; Table 3 lists the complete configuration. In each run the model state at the epoch with the highest validation macro-F1 is saved and restored before the test split is evaluated, and the test split is evaluated exactly once per run. The test split is never used for a decision.

3 Implementation Results

3.1 Training Behaviour

Selected checkpoints. For the baseline the selected epoch is 26, 25, and 30 of 30: for seeds 42 and 67 validation accuracy peaks at epochs 26 and 25 and then declines to 0.3340 and 0.3443 by epoch 30, while the minimum validation loss occurs at epochs 12 and 12. For the frozen configuration the selected epoch is 30/30 in all three seeds, and both validation loss and validation accuracy improve monotonically through the final epoch. For full fine-tuning the selected epoch is 4, 9, and 4 of 30.

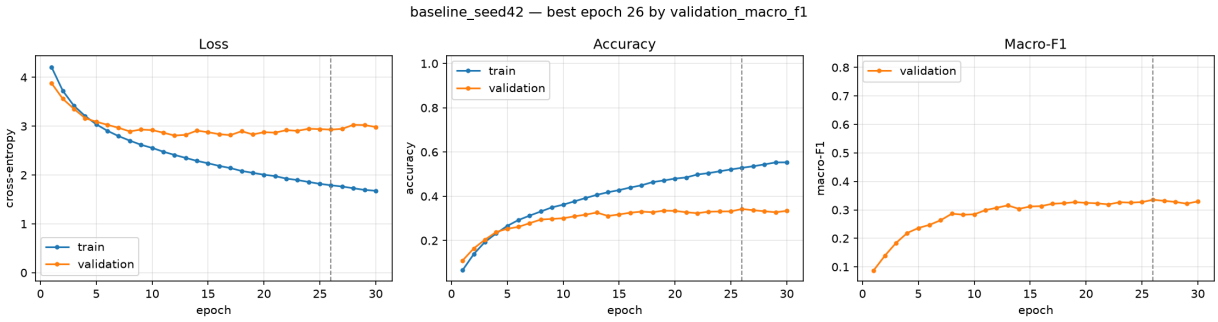
Overfitting. Full fine-tuning overfits inside the fixed budget. Training accuracy reaches 0.9879–0.9889 at epoch 30, whereas validation accuracy peaks at 0.8033–0.8049 between epochs 4 and 9, and validation loss rises from approximately 0.77 at its minimum to 1.048–1.078 at

Table 3: Training configuration for all nine runs (handbook §3.2).

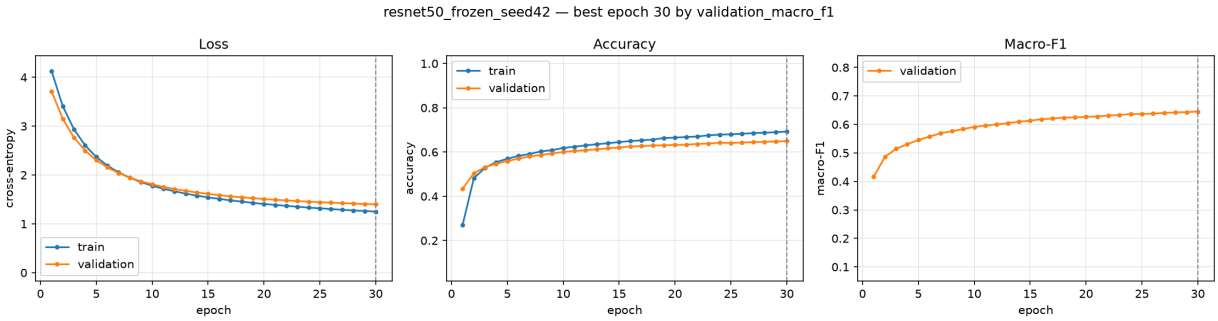
Setting	From-scratch CNN	ImageNet ResNet-50 (full/frozen)
Initialisation	random	ImageNet `IMAGENET1K_V2`
Input resolution	128×128	224×224
Optimiser	AdamW	AdamW
Learning rate	10^{-3}	10^{-4}
Weight decay	10^{-4}	10^{-4}
Batch size	128 (train, `drop_last`)	128 (train, `drop_last`)
Epochs	30	30
LR scheduler	none	none
Regularisation	weight decay only; dropout disabled	weight decay only
Early stopping	none	none
Checkpoint criterion	best validation macro-F1	best validation macro-F1
Training seeds	42, 67, 69	42, 67, 69
Split seed	42	42
Mixed precision	enabled (CUDA autocast + gradient scaler)	enabled
Hardware	1× NVIDIA A100-SXM4-80GB	1× NVIDIA A100-SXM4-80GB
Software	PyTorch 2.11.0+cu128, CUDA 12.8, torchvision 0.26.0	PyTorch 2.11.0+cu128, CUDA 12.8, torchvision 0.26.0
Hyperparameter selection	fixed a priori; no search performed	fixed a priori; no search performed

Table 4: Training behaviour per run. "Best ep." is the epoch selected by validation macro-F1. "Val acc" and "Val F1" are validation accuracy and validation macro-F1 at that epoch. "Train acc" is training accuracy at epoch 30. "Val loss (min)" is the minimum validation loss, with the epoch in parentheses. "Val loss (end)" is validation loss at epoch 30. All accuracies and losses are fractions.

Model	Seed	Best ep.	Val acc	Val F1	Train acc	Val loss (min)	Val loss (end)
Baseline CNN	42	26	0.3424	0.3356	0.5534	2.809 (12)	2.984
	67	25	0.3691	0.3648	0.5826	2.692 (12)	3.000
	69	30	0.3815	0.3763	0.5949	2.592 (15)	2.825
ResNet-50 frozen	42	30	0.6492	0.6453	0.6929	1.399 (30)	1.399
	67	30	0.6506	0.6463	0.6915	1.399 (30)	1.399
	69	30	0.6484	0.6445	0.6911	1.399 (30)	1.399
ResNet-50 full	42	4	0.8042	0.8032	0.9880	0.763 (4)	1.048
	67	9	0.8049	0.8053	0.9889	0.771 (4)	1.072
	69	4	0.8033	0.8030	0.9879	0.768 (4)	1.078



(a) Baseline CNN, seed 42



(b) ResNet-50 frozen, seed 42



(c) ResNet-50 full fine-tuning, seed 42

Figure 2: Training and validation loss, accuracy, and validation macro-F1 for seed 42. The dashed vertical line marks the epoch selected by validation macro-F1. Panels (b) and (c) share the data, the augmentation, and the epoch budget and differ only in whether the backbone parameters are trainable.

epoch 30 (Table 4, Figure 2). The training–validation accuracy gap at epoch 30 is 0.186–0.192. No scheduler, early stopping, label smoothing, or additional regularisation is configured, so the selection of the best-validation checkpoint is the only mechanism that limits the effect of this overfitting on the reported test numbers.

The baseline and the frozen configuration do not show this pattern. Their training–validation accuracy gaps at epoch 30 are 0.213–0.238 and 0.041–0.044 respectively. For the baseline the minimum validation loss (epochs 12–15) and the maximum validation accuracy (epochs 25–30) occur at different epochs, so the two selection criteria disagree inside the budget. For the frozen configuration both criteria improve monotonically to the final epoch, so the budget terminates training before either criterion reaches a maximum.

Training time. Per-epoch wall-clock time averaged 30.2 s, 30.4 s, and 73.6 s for the three baseline runs; 43.9 s, 42.0 s, and 38.7 s for the frozen runs; and 65.8 s, 53.4 s, and 50.6 s for the fine-tuning runs.

Implementation issues and fixes.

- *BatchNorm statistics under a frozen backbone.* With the backbone frozen, BatchNorm running statistics would still be updated in training mode, which changes the feature extractor during training. Fixed by switching every `BatchNorm2d` module to evaluation mode on each epoch when the backbone is frozen.
- *Truncated JPEG files.* Decoding the release emits `Truncated File Read` warnings for a small number of files. Fixed by filtering this specific warning message in the loader.
- *GPU memory across nine sequential runs.* Each model is deleted and the CUDA cache is emptied between runs so that all configurations fit on a single GPU.
- *Training-time variance.* The baseline seed-69 run recorded 2,208.3 s against 907.3 s and 911.8 s for seeds 42 and 67 with an identical epoch count, batch size, and hardware; the mean baseline training time is affected by this single run.
- *Two data paths.* The training pipeline reads the HuggingFace parquet mirror, whereas the EDA and near-duplicate notebooks read the `torchvision` release. Both reproduce the same official split, but the image ordering differs, so the duplicate indices produced by the EDA notebook are not directly transferable to the training pipeline.

3.2 Quantitative Results

Table 5: Test and validation results, mean ± standard deviation over training seeds 42, 67, and 69. "Best epoch" is the mean selected epoch over the three seeds. Trainable parameter counts are given in Table 8.

Model	Test acc. (%)	Test macro-F1 (%)	Val. acc. (%)	Val. macro-F1 (%)	Best epoch
Baseline CNN	40.18 ± 2.38	39.74 ± 2.48	36.44	35.89	27.0
ResNet-50 frozen	69.29 ± 0.06	68.96 ± 0.08	64.94	64.54	30.0
ResNet-50 full	84.20 ± 0.10	84.19 ± 0.15	80.41	80.38	5.7

Accuracy versus macro-F1. Because the class distribution is balanced, test accuracy and test macro-F1 differ by at most 0.45 percentage points in any configuration (baseline 0.45, frozen 0.33, full fine-tuning 0.01).

Test–validation ordering. In all nine runs the test accuracy exceeds the validation accuracy of the same run: by 3.3, 3.9, and 4.0 percentage points for the baseline; by 4.4, 4.2, and 4.4 points for the frozen configuration; and by 3.7, 3.8, and 3.9 points for full fine-tuning. The validation split is a subset of the official training split, whose labels are documented as automatically assigned and intentionally uncleaned, whereas the official test labels are manually verified (§1.4). The consistent sign and the narrow magnitude range of the difference across all three architectures is

Table 6: Per-run validation and test metrics with compute times. Accuracy and macro-F1 are fractions; training time is in seconds and inference time is in milliseconds per image.

Model	Seed	Val. acc.	Val. F1	Test acc.	Test F1	Training (s)	Inference (ms/image)
Baseline CNN	42	0.3424	0.3356	0.3755	0.3698	907.3	0.438
	67	0.3691	0.3648	0.4081	0.4044	911.8	0.417
	69	0.3815	0.3763	0.4219	0.4179	2208.3	0.393
ResNet-50 frozen	42	0.6492	0.6453	0.6935	0.6905	1327.2	0.594
	67	0.6506	0.6463	0.6925	0.6891	1264.2	0.818
	69	0.6484	0.6445	0.6927	0.6892	1164.5	0.779
ResNet-50 full	42	0.8042	0.8032	0.8410	0.8402	1975.4	0.612
	67	0.8049	0.8053	0.8428	0.8432	1601.7	0.669
	69	0.8033	0.8030	0.8422	0.8423	1517.2	0.586

consistent with this difference in label quality rather than with a difference in image difficulty between the splits.

Per-class results. Per-class precision, recall, and F_1 were computed for all 101 classes in all nine runs. For full fine-tuning the lowest mean F_1 over the three seeds is `steak` (0.553), followed by `pork_chop` (0.638), `apple_pie` (0.645), `filet_mignon` (0.661), `chocolate_mousse` (0.686), `huevos_rancheros` (0.690), `foie_gras` (0.691), `bread_pudding` (0.694), `ravioli` (0.703), `tuna_tartare` (0.733), `omelette` (0.735), and `crab_cakes` (0.737). The highest are `edamame` (0.993), `macarons` (0.972), `oysters` (0.958), `deviled_eggs` (0.941), `seaweed_salad` (0.941), and `spaghetti_carbonara` (0.940). Every class in the lowest group belongs to a set of visually similar dishes; the classes with the highest F_1 have a distinctive colour and shape.

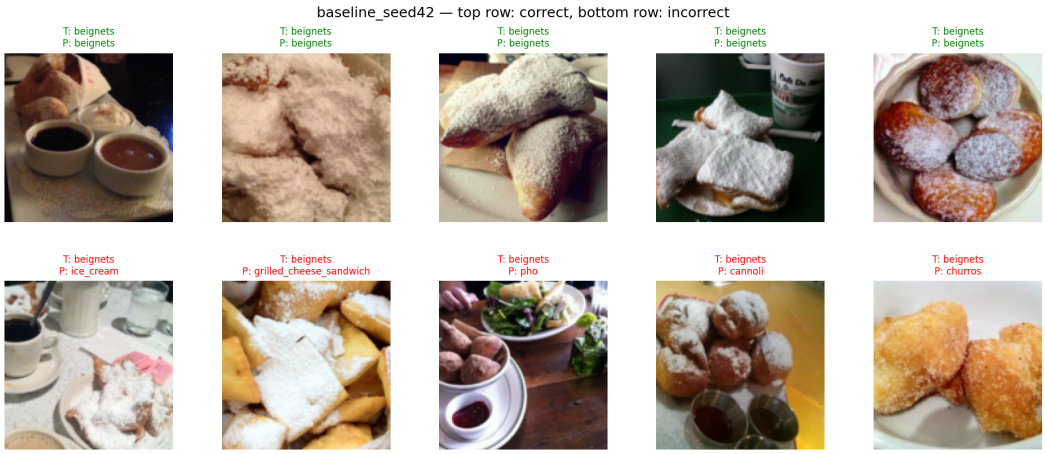
3.3 Qualitative Results

Figure 3 records, for seed 42, five correct and five incorrect predictions from the first test batch for the baseline and for full fine-tuning; the pipeline stores the equivalent grid for all nine runs. The grids sample one batch of 128 test images and are therefore illustrative. The qualitative evidence that covers the whole test split is the per-class metric table of §3.2 and the confusion analysis of §3.5, both computed over all 25,250 test images. Difficult-but-correct cases were not enumerated separately by the pipeline: the prediction grids are built from the first five correct and the first five incorrect predictions of the batch, not from a difficulty ranking, and the stored confusion matrices and per-class metrics support the identification of such cases after the run rather than during it.

3.4 Comparison and Discussion

Full fine-tuning exceeds the frozen backbone by 14.91 percentage points of test accuracy (84.20% versus 69.29%) and 15.23 points of test macro-F1 (84.19% versus 68.96%). The difference is approximately 157 times the larger of the two across-seed standard deviations (0.10 and 0.06 points), and the per-seed ranges do not overlap: full fine-tuning spans 84.10–84.28% and the frozen configuration spans 69.25–69.35%. The hypothesis of Table 2 is supported.

The baseline is 29.11 points below the frozen configuration and 44.02 points below full fine-tuning in test accuracy. Its measured inference latency (0.438, 0.417, and 0.393 ms per image) is lower than that of both ResNet-50 configurations (0.594–0.818 ms frozen, 0.586–0.669 ms full), although that comparison is not resolution-matched: the baseline reads 128×128 inputs while both ResNet-50 configurations read 224×224 inputs. Across the three configurations inference



(a) Baseline CNN, seed 42



(b) ResNet-50 full fine-tuning, seed 42

Figure 3: Test predictions versus ground truth for seed 42. Top row of each panel: five correct predictions. Bottom row: five incorrect predictions. Each title gives the true label (T) and the predicted label (P).

latency spans a factor of 1.75 while the parameter count spans a factor of 2.7; in the baseline, 8.44 M of its 8.83 M parameters (95.6%) are in its two dense head layers, so its parameter count does not reflect convolutional depth.

Indicated from the recorded data, there are three possible factors contributing to the difference between the frozen and the fully fine-tuned configuration.

1. *Representation adaptation.* At equal model size, data, augmentation, optimiser, learning rate, batch size, and the given epoch budget, allowing the backbone to adapt raises test accuracy by 14.91 points. On seed 42 the number of test errors falls from 7,738 (accuracy 0.6935) to 4,016 (accuracy 0.8410). The frozen configuration is a linear probe on ImageNet features, so the gap measures the extent to which the ImageNet representation may not be linearly sufficient for fine-grained dish classification. However, because the frozen configuration has not converged, the experiment does not establish the maximum accuracy attainable with fixed features or isolate the contribution of representation adaptation from optimization effects.
2. *Optimisation budget.* The two configurations are not equally converged. The frozen configuration selects its final epoch (30/30) in all three seeds and its validation curves are still improving at that point, whereas full fine-tuning selects epoch 4–9 and then degrades. The epoch budget is formally a fixed factor but truncates the frozen configuration while it is still improving, so part of the 14.91 points reflects the shared budget rather than the frozen/trainable distinction alone.
3. *Generalisation behaviour.* Full fine-tuning reaches a training–validation accuracy gap of 0.186–0.192 at epoch 30, against 0.041–0.044 for the frozen configuration. The additional trainable capacity is used to fit the training split, so the gap between the two configurations is not attributable to under-fitting of the frozen model alone.

The gap between the baseline and both ResNet-50 configurations mixes four factors (pretraining, input resolution, learning rate, and parameter count), so the recorded data do not attribute it to any single factor.

Conclusions supported by the recorded data: ImageNet pretraining with full fine-tuning reaches 84.20% test accuracy on Food-101 within 30 epochs on a single A100; the same backbone restricted to a linear probe reaches 69.29%; a self-implemented 8.83 M-parameter convolutional network trained from scratch within the same budget reaches 40.18%; and the ordering full > frozen > from-scratch holds in each of the three seeds.

Conclusions not supported by the recorded data: whether full fine-tuning with a learning-rate schedule, or with a smaller epoch budget, performs better than the 30-epoch run reported here; whether another backbone performs better, since only ResNet-50 was evaluated; whether the input resolution or a change of augmentation policy changes the ordering, since neither was varied; and what fraction of the baseline gap is due to pretraining as opposed to resolution, learning rate, or parameter count, since those factors were not separated.

3.5 Error Analysis

On seed 42, full fine-tuning produces 4,016 test errors (accuracy 0.8410), the frozen configuration produces 7,738 (0.6935), and the baseline produces 15,768 (0.3755). The twenty most frequent confusion pairs of full fine-tuning (Figure 4) account for 432 of its 4,016 errors, i.e. 10.8%; the remaining 89.2% are distributed over the other off-diagonal cells of the 101×101 confusion matrix. Table 7 groups these twenty pairs by the semantics of the confused classes. Frequencies are test images per pair on seed 42; mean F_1 is over seeds 42, 67, and 69.

Comparison across configurations. The frozen configuration and full fine-tuning confuse the same class families. Three of the frozen configuration's five most frequent pairs (Figure 5) belong to the red-meat and raw-preparation groups of Table 7 (`steak`→`filet_mignon` 44,

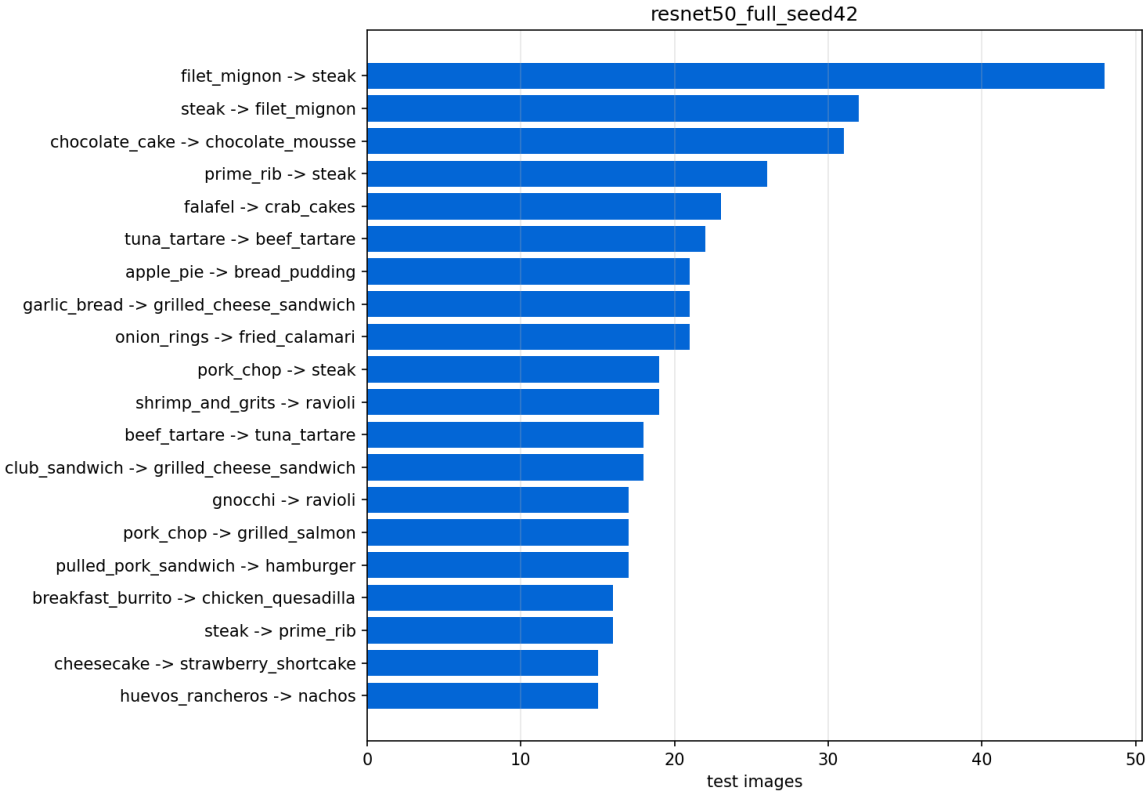


Figure 4: The twenty most frequent confusion pairs (true → predicted) of full fine-tuning on the 25,250-image test split, seed 42.

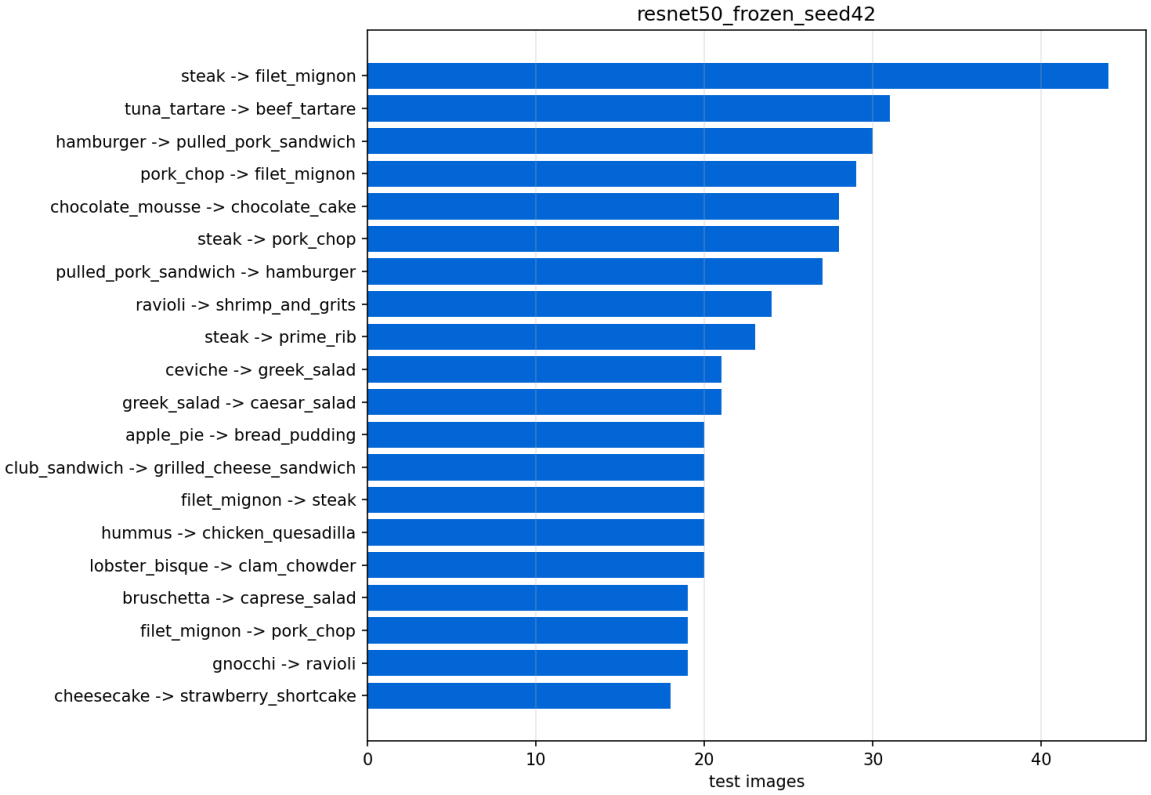


Figure 5: The twenty most frequent confusion pairs (true → predicted) of the frozen backbone on the same test split, seed 42.

Table 7: Error taxonomy for full fine-tuning.

Group	Representative pairs (true → predicted)	Freq.	Mean F_1	Hypothesised cause
Red-meat cuts	`filet_mignon`→`steak` 48; `steak`→`filet_mignon` 32; `prime_rib`→`steak` 26; `pork_chop`→`steak` 19; `steak`→`prime_rib` 16	141	0.553–0.828	Same cut of meat with similar garnish and plate; the class boundary is a property of the cut and the cooking method, not of the image
Chocolate desserts	`chocolate_cake`→`chocolate_mousse` 31	31	0.686–0.751	Identical colour and texture; both are plated as a dark brown mass
Raw preparations	`tuna_tartare`→`beef_tartare` 22; `beef_tartare`→`tuna_tartare` 18	40	0.733–0.830	Same moulded-disc plating and comparable colour; the classes differ by the protein only
Battered fried items	`falafel`→`crab_cakes` 23; `onion_rings`→`fried_calamari` 21	44	0.737–0.903	Golden-brown fried pieces of similar scale, not separable by colour or shape at 224×224
Filled pasta and starch	`shrimp_and_grits`→`ravioli` 19; `gnocchi`→`ravioli` 17	36	0.703–0.775	Small pale starch pieces in a light sauce; the shape distinction occupies few pixels
Sandwiches	`garlic_bread`→`grilled_cheese_sandwich` 21; `club_sandwich`→`grilled_cheese_sandwich` 18; `pulled_pork_sandwich`→`hamburger` 17	56	0.763–0.891	Bread-plus-filling composition; the discriminating content occupies a small region of the image
Egg breakfast dishes	`breakfast_burrito`→`chicken_quesadilla` 16; `huevos_rancheros`→`nachos` 15	31	0.690–0.832	Shared tortilla, egg, and salsa components; the class is defined by the combination rather than by a single visible element
Cakes and puddings	`apple_pie`→`bread_pudding` 21; `cheesecake`→`strawberry_shortcake` 15	36	0.645–0.846	Dessert with topping and syrup; the pastry base and the fruit layer are visually interchangeable
Pork chop versus salmon	`pork_chop`→`grilled_salmon` 17	17	0.638	Grilled protein with sear marks on a plate; hue and grill pattern dominate over texture
Total (twenty pairs)		432		

tuna_tartare→beef_tartare 31, pork_chop→filet_mignon 29), and its fifth most frequent pair is the reciprocal of the chocolate group (chocolate_mousse→chocolate_cake 28). For the pairs observable in both lists the count falls under full fine-tuning (steak→filet_mignon 44→32, tuna_tartare→beef_tartare 31→22). Full fine-tuning therefore reduces the frequency of these errors without changing which classes are confused, so the residual errors correspond to properties of the label definition rather than to the capacity of the model.

The baseline's top eight confusion pairs, with counts, are samosa→spring_rolls (38), pad_thai→fried_rice (36), chocolate_cake→chocolate_mousse (34), steak→filet_mignon (34), steak→baby_back_ribs (33), spaghetti_carbonara→fried_rice (32), filet_mignon→steak (28), and greek_salad→caprese_salad (27). Five of these are sibling-class confusions that also appear in the fine-tuned model (chocolate_cake→ chocolate_mousse, steak→filet_mignon, steak→baby_back_ribs, filet_mignon→steak, greek_salad→caprese_salad), and two map a noodle or pasta dish onto a rice dish (pad_thai→fried_rice, spaghetti_carbonara→fried_rice). Neither rice mapping appears in the twenty most frequent pairs of either ResNet-50 configuration. The baseline therefore produces coarse, cross-category errors in addition to the fine-grained errors of the two ResNet-50 configurations, whereas the ResNet-50 configurations produce fine-grained errors only.

Improvement directions indicated by the recorded data. The 0.186–0.192 training–validation gap of full fine-tuning corresponds to the absence of a learning-rate schedule and of early stopping, which the configuration records as not used; the selection of epoch 4–9 of 30 already bounds the effect of that gap on the reported numbers. The monotone validation curves of the frozen configuration at epoch 30 correspond to an epoch budget that terminates the linear probe before convergence. The two groups whose discriminating detail is small relative to the image (filled pasta and starch, battered fried items) correspond to an input resolution of 224×224, and a resolution sweep above 224 is the experiment that would test this. None of these three interventions was evaluated in this assignment.

3.6 Ablation Study

The mandatory controlled factor study is the frozen-versus-full fine-tuning ablation specified in Table 2 and reported in Tables 5 and 6. The result is a 14.91-point test-accuracy difference and a 15.23-point macro-F1 difference in favour of full fine-tuning, with the same sign and a comparable magnitude in all three seeds, and across-seed standard deviations of 0.10 and 0.06 points. The frozen configuration trains 206,949 of 23,714,981 parameters (0.87%), so the ablation also quantifies the cost of the alternative: 14.91 points of test accuracy for 23,508,032 additional trainable parameters.

3.7 Compute-Cost Analysis

Table 8: Compute cost per configuration. Times are means over the three seeds on one NVIDIA A100-SXM4-80GB.

Model	Total parameters	Trainable parameters	Train time (s)	Inference (ms/image)
Baseline CNN	8,829,349	8,829,349	1,342.5	0.416
ResNet-50 frozen	23,714,981	206,949	1,252.0	0.730
ResNet-50 full	23,714,981	23,714,981	1,698.1	0.623

Training time is wall-clock time for 30 epochs and includes the validation pass after each epoch. The baseline mean is affected by one run at 2,208.3 s against 907.3 s and 911.8 s for its other two seeds; excluding that run the baseline mean is 909.6 s. The frozen configuration is the cheapest to train (1,252.0 s) and full fine-tuning the most expensive (1,698.1 s, i.e. 35.6% above frozen) for

14.91 points of additional test accuracy. Inference time is measured over the full 25,250-image test split after two warm-up batches on the same GPU, without mixed precision. The two ResNet-50 configurations share the same network and the same computation at inference, so the difference between their measured inference times (0.730 and 0.623 ms/image) is measurement variability and not a property of the freeze setting.

3.8 Reproducibility

Table 9 records the computational environment, execution procedure, and committed artifacts that reproduce all nine runs. The training configuration itself is in Table 3 and the split sizes in Table 1.

Table 9: Reproducibility settings and experimental artifacts.

Aspect	**Configuration / Details**
Hardware	1× NVIDIA A100-SXM4-80GB GPU.
Software environment	PyTorch 2.11.0+cu128, CUDA 12.8, torchvision 0.26.0. The remaining dependencies are pinned in `assignment2/requirements.txt`.
Dataset	Food-101 is downloaded on first use and is not committed. The training pipeline reads the `ethz/food101` parquet mirror in `data/food-101/`; the EDA and duplicate notebooks use the `torchvision` release in `data/asm2/`.
Documentation	Installation, dataset preparation, training and evaluation commands are documented in `assignment2/README.md`.
Execution	Install with `pip install -r assignment2/requirements.txt`, then execute all cells of `assignment2/notebooks/main.ipynb` with seeds `[42, 67, 69]`, split seed `42`, `EPOCHS = 30`, `BATCH_SIZE = 128` and `LIMIT = None`. The notebook rebuilds the split, trains all nine runs, restores each best-validation checkpoint, evaluates the test split once per run, and rewrites every artifact below.
Checkpoints and artifacts	Checkpoints are stored in `assignment2/checkpoints/`. Results and supporting artifacts are stored in `assignment2/results/`: CSV and JSON results, per-epoch training histories, curves, prediction examples, and full confusion matrices. A checkpoint is restored by rebuilding the architecture with `build_model()` and the loaders with `build_food101_loaders()`, then loading its `model_state_dict`.
Analysis notebooks	`assignment2/notebooks/eda.ipynb` and `assignment2/notebooks/check_duplicate.ipynb`; the duplicate analysis operates on all 101,000 images and writes its candidate list to `data/asm2/`, so it does not alter the split used by the nine runs.
Provenance and AI usage	The AI-use record is `assignment2/AI_USAGE.md`. The assignment page is `assignment2/index.html`; this report is `assignment2/materials/G-03_A2_Report.pdf`.

3.9 Limitations and Conclusion

- **Data limits.** One dataset and one track. The official training labels are documented as noisy, which bounds the attainable validation accuracy and accounts for the systematic test-above-validation ordering reported in §3.2. The 102 train–test near-duplicate pairs identified by the pHash analysis (at most 0.40% of the test split) were not removed from the split used for training.
- **Method limits.** One pretrained backbone (ResNet-50); a single global learning rate; no scheduler, early stopping, label smoothing, mixup, cutmix, or dropout in the baseline; no hyperparameter search.

- **Experimental limits.** One controlled factor. Input resolution, augmentation, loss, and backbone were not varied, so no statement about them can be made. Three seeds characterise seed noise for these configurations but not hyperparameter noise. The epoch budget terminates the frozen configuration while both validation criteria are still improving, and overfits the fully fine-tuned configuration after epoch 4–9.
- **Generalisability.** The results are specific to Food-101, to the ImageNet-pretrained ResNet-50 family, and to a 30-epoch single-GPU budget. Extension to other fine-grained datasets, other backbones, or other compute budgets is not established.
- **Future work.** A learning-rate schedule with early stopping to act on the 0.186–0.192 training–validation gap; a resolution sweep above 224×224; a frozen configuration trained to convergence; a second backbone as a backbone A-versus-B factor; and applying the duplicate-removal list to the training split.
- **Answer to the research question.** On Food-101, within a fixed 30-epoch budget on one A100-SXM4-80GB, an ImageNet ResNet-50 fully fine-tuned reaches $84.20\% \pm 0.10$ test accuracy and $84.19\% \pm 0.15$ test macro-F1; a frozen-backbone linear probe on the same model reaches $69.29\% \pm 0.06$ and $68.96\% \pm 0.08$; and a self-implemented 8.83 M-parameter CNN trained from scratch reaches $40.18\% \pm 2.38$ and $39.74\% \pm 2.48$. Allowing the backbone to adapt is worth 14.91 points of test accuracy and 15.23 points of macro-F1 over training the head alone, at 23,508,032 additional trainable parameters (99.13% of the model) and 35.6% more training time.

AI Usage Disclosure

AI tools were used in this assignment as assistants under member supervision. The disclosure follows handbook §5.2. The complete log, with affected files and verification records, is in `assignment2/AI_USAGE.md`; the same disclosure is maintained on the course landing page and on the Assignment 2 page, as required by handbook §5.1.

- **Tools and versions:** opencode CLI (GLM-5.3); ChatGPT-4; ChatGPT-6; Gemini (model unverified).
- **Members:** Nguyen Van An used opencode CLI (GLM-5.3) for all Assignment 2 entries; Vo Le Hai Dang used ChatGPT-4 for the shared landing page HTML scaffolding (root `AI_USAGE.md`, entry 1), which the Assignment 2 page links to; Huynh Vuong Khang used Gemini and ChatGPT-6 to implement perceptual hashing for near-duplicate detection.
- **Stages:** A2-M1 dataset proposal, 06 Oct 2026; A2-M2 duplicate detection code, 25, and pipeline reuse, 26–27 Oct 2026; A2-M3 experiment plumbing, 28 Oct 2026; landing page scaffolding, September 2026.
- **Purpose and affected sections:** drafting the dataset proposal (§1.2, §1.4); implementing and debugging the algorithm for hashing and indexing near-duplicate pairs (§1.3); reusing the Assignment 1 pipeline and adding the Food-101 loader and the `torchvision` ResNet-50 adaptor (§2.1–§2.3); organising the seed loop and formatting the per-class metrics, the confusion extraction, and the ablation summary (§3.2–§3.6); the HTML structure of the shared pages (front matter).
- **Representative prompts:** a request to identify a dataset and check it against the handbook thresholds; a request to find a duplicate detection method; a request to debug incorrect indexing of the given implementation; a request that Assignment 2 follow the Assignment 1 structure and reuse its pipeline; a request to extend the M2 pipeline to the M3 multi-seed and ablation setup; a request for a basic HTML layout for the landing page and assignment pages.
- **Editing and verification:** dataset facts (class counts, split sizes, licence text) were quoted from the official dataset card and the ETH Zürich dataset page before adoption; online sources and manual checks for detected duplicates; the reused components were reviewed by the member; the reported accuracy, macro-F1, training time, and inference time were checked against `results/summary.csv` and `results/results.json`; the full 30-epoch, three-seed experiment was executed by the member; the HTML structure was reviewed and adjusted against handbook §2.1–§2.2.
- **Responsible member for final verification:** Nguyen Van An (Assignment 2 technical content); Vo Le Hai Dang (landing page); Huynh Vuong Khang (Assignment 2 EDA).
- **Sources used for verification:** CO3133 handbook §17–§23 and §2.1–§5.2; the Food-101 dataset

card and the official ETH Zürich dataset page; `torchvision` ResNet documentation; PyTorch API documentation; the Wikipedia article on Perceptual hashing.